

CSE406 · ADVANCE JAVA PROGRAMMING · LECTURE

Collections, Streams, and Filters in Java

Builder Pattern · Lambdas · Filtering · Method Chaining · Pipelines · Parallel Streams · Reduction

A Beginner-to-Advanced Course for B.Tech CSE · BCA · MCA Students

Course Instructor: Dr. B. Moharana

WHAT WE WILL COVER

Roadmap of this Course

1. Java Collection Framework basics
2. Arrays vs Collections
3. Why Java 8 introduced Streams
4. Builder Pattern & method chaining
5. Introduction to the Stream interface
6. Lambda expressions
7. Iterating collections (4 ways)
8. Filtering with filter() & Predicate
9. Chaining multiple filters
10. map() and transforming data
11. The Stream pipeline: source → intermediate → terminal
12. Intermediate operations (sorted, distinct, limit, skip)
13. Terminal operations (collect, count, reduce, match)
14. Parallel Streams
15. Decomposing & merging work (Fork/Join)
16. Reduction & the reduce() method
17. A complete real-world pipeline example
18. Performance: sequential vs parallel
19. Interview questions
20. Summary mind map
21. 15 practice programs

1

SECTION 1

Introduction to Collections

Setting the foundation before we touch Streams

FOUNDATION

What is the Java Collection Framework?

- 1 A unified architecture**

A set of classes and interfaces (in `java.util`) to store, manage, and process groups of objects as a single unit.
- 2 Not just one class**

It includes `List`, `Set`, `Queue`, `Map` and their many implementations like `ArrayList`, `HashSet`, `LinkedList`, `HashMap`.
- 3 Ready-made data structures**

Instead of writing your own linked list or dynamic array, Java gives you tested, optimized versions to use directly.
- 4 The base for Streams**

Every Stream in Java 8 usually begins its life from a Collection — so Collections come first, Streams come next.

Teaching tip: Draw a simple box on the whiteboard labeled 'java.util' and write `List`, `Set`, `Queue`, `Map` inside it. Tell students: "Think of the Collection Framework as a toolbox — different tools (data structures) for different jobs."

THE PROBLEM WITH ARRAYS

Why Are Collections Needed?

Arrays (Old Way)

- Fixed size — decided at creation time
- Cannot easily insert/delete in the middle
- Only store one data type
- No built-in search, sort, or utility methods
- Manual index management for everything

Collections (Modern Way)

- Grow and shrink automatically (dynamic size)
- Easy insert, delete, and reorder operations
- Can store objects with rich behavior
- Built-in methods: add(), remove(), sort(), contains()
- Designed to work seamlessly with Streams

EVERYDAY ANALOGIES

Real-Life Examples of Collections



List

A shopping list — items in order, duplicates allowed (milk, milk, bread)



Set

Unique roll numbers in a class — no two students share the same number



Queue

A ticket counter line — first person in line is served first (FIFO)



Map

A phone directory — each name (key) is linked to one phone number (value)

SIDE-BY-SIDE COMPARISON

Arrays vs Collections

Array

- Fixed length once created
- Can hold primitives (int, char) directly
- Faster for simple, fixed-size data
- Very few built-in methods
- length property (not a method)

Collection

- Grows/shrinks dynamically
- Only holds objects (Integer, not int)
- Slightly more overhead, far more flexible
- Rich set of utility methods
- size() method call

THE FOUR MAIN FAMILIES

Types of Collections



List

Ordered, allows duplicates. E.g.
ArrayList, LinkedList



Set

No duplicates allowed. E.g.
HashSet, TreeSet



Queue

Processes elements in order
(FIFO). E.g. LinkedList,
PriorityQueue

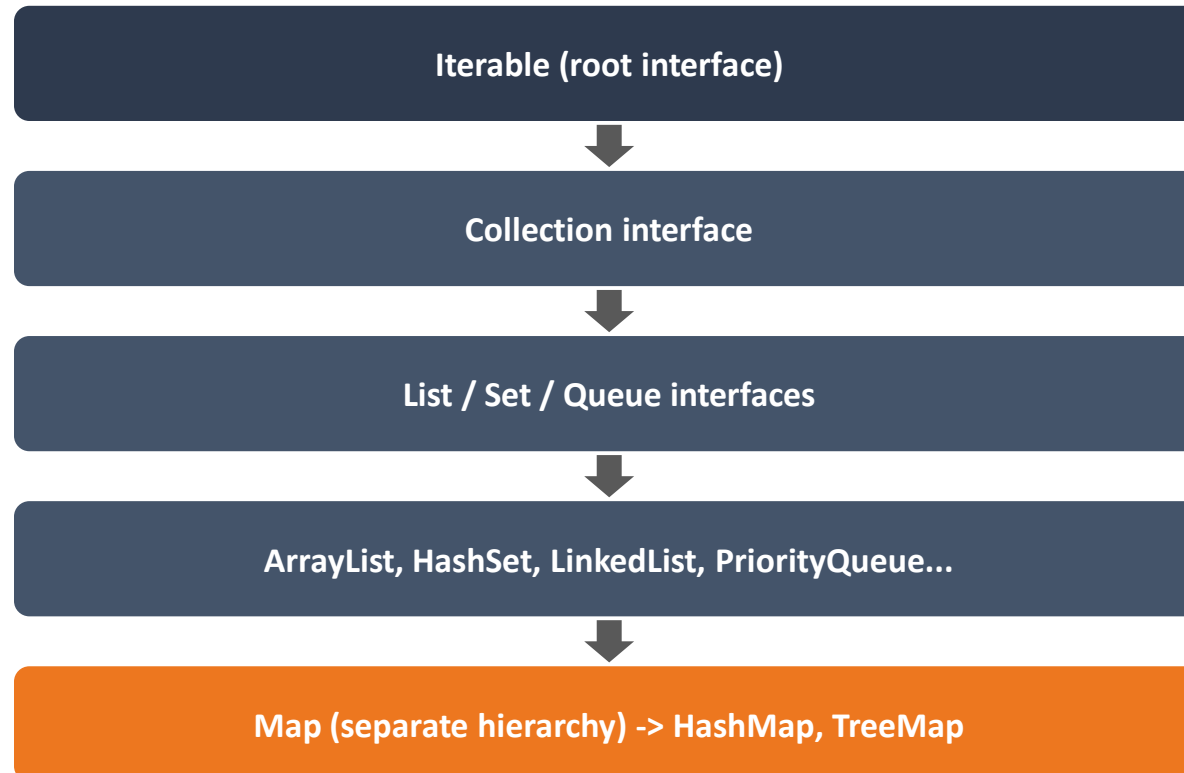


Map

Key-value pairs (not a true
Collection, but part of the
framework). E.g. HashMap,
TreeMap

HOW IT ALL FITS TOGETHER

The Collection Hierarchy



This is a simplified view of the hierarchy.

Every List, Set, and Queue ultimately extends the Collection interface, which extends Iterable -- this is WHY every collection can be used in a for-each loop and with Streams.

Map is intentionally separate because it stores pairs of values (key + value), not single elements.

2

SECTION 2

What Is a Collection?

Definition, advantages, and our first program

DEEP DIVE

Collection: Definition & Advantages

- 1** **Definition**

A Collection is an object that groups multiple elements into a single unit, so we can add, remove, and process them together.
- 2** **Dynamic sizing**

No need to declare a size upfront -- the collection grows and shrinks automatically as you add or remove data.
- 3** **Rich API**

Comes with ready-made methods: `add()`, `remove()`, `contains()`, `size()`, `sort()`, and many more.
- 4** **Real-world use**

Used everywhere: storing user lists, shopping cart items, search results, database query results, and more.

HANDS-ON

Program 1: Creating Your First Collection

Let's store three student names using an ArrayList -- the most commonly used List implementation.

```
import java.util.*;

public class Demo1 {
    public static void main(String args[]) {
        ArrayList<String> students = new ArrayList<>();

        students.add("Raju");
        students.add("Amit");
        students.add("Neha");

        System.out.println(students);
    }
}
```

OUTPUT

```
[Raju, Amit, Neha]
```

KEY POINTS

- ▶ `import java.util.*;` gives access to `ArrayList`
- ▶ `<String>` is a Generic -- it locks the list to only hold `String` values
- ▶ `add()` inserts an element at the end
- ▶ `println(students)` automatically calls `toString()`

3

SECTION 3

Why Java 8 Introduced Streams

The problem with pre-Java-8 code

THE OLD WAY WAS PAINFUL

Problems Before Java 8

- 1 Too many loops**

Every single task (filter, transform, sum) needed its own explicit for-loop, written from scratch each time.
- 2 Temporary collections everywhere**

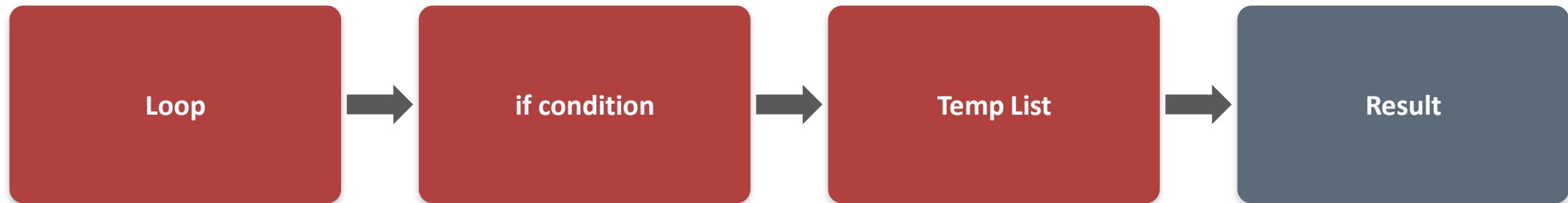
Programmers had to create extra ArrayLists just to hold intermediate results between steps.
- 3 Difficult, repetitive code**

The same loop-and-if pattern was copy-pasted again and again across projects.
- 4 Hard to read at a glance**

A reader had to mentally execute the whole loop to understand WHAT the code was trying to do.

BEFORE AND AFTER JAVA 8

Traditional Approach vs Stream Approach

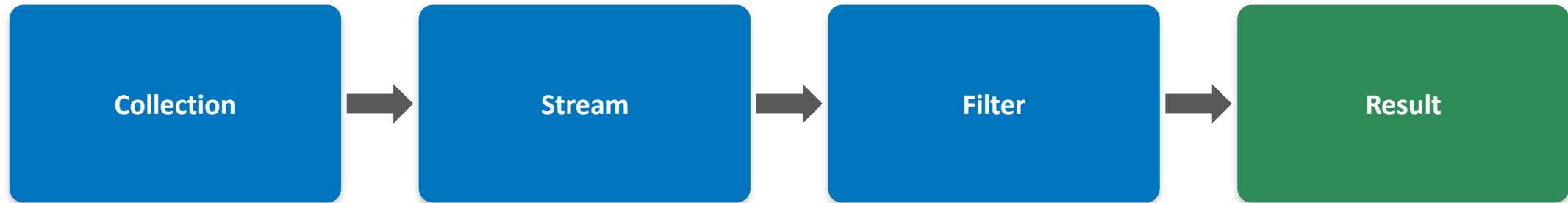


Traditional Approach: we manually loop through the collection, check a condition with if, store matching items in a new temporary list, then use that list as our result. Every step is manual and verbose.

Compare this on the next slide with the Stream approach -- Collection -> Stream -> Filter -> Result -- which expresses the SAME logic in a single readable line of code.

ONE CLEAN PIPELINE

The Stream Way



Notice how the Stream approach reads almost like English: "Take the Collection, turn it into a Stream, filter it, get the Result."

This is the core idea of Streams: instead of describing HOW to loop, we describe WHAT we want -- this style is called declarative programming.

4

SECTION 4

The Builder Pattern

Method chaining -- the idea behind every Stream pipeline

A DESIGN PATTERN FOR CLEAN CODE

What Is the Builder Pattern?

- 1 Definition**

A design pattern where an object is built step-by-step, and each step returns the SAME object so you can keep calling more methods on it.
- 2 Method Chaining**

Calling one method after another on the same line, connected with dots:
`obj.methodA().methodB().methodC()`
- 3 Why it's useful**

It avoids repeating the object's name again and again, and reads like a clear sequence of steps.
- 4 Connection to Streams**

Every Stream operation (filter, map, sorted) returns a Stream -- so we can keep chaining, exactly like the Builder Pattern.

PROGRAM

Builder Pattern in Action: StringBuilder

StringBuilder is a perfect real Java example of the Builder Pattern you may have already used.

```
StringBuilder sb = new StringBuilder();

sb.append("Hello")
  .append(" Java")
  .append(" Students");

System.out.println(sb);
```

OUTPUT

```
Hello Java Students
```

KEY POINTS

- ▶ `append()` adds text AND returns the same `StringBuilder` object
- ▶ That returned object lets us call `.append()` again immediately
- ▶ This chain of calls is called Method Chaining
- ▶ Streams use this exact same trick to chain `filter/map/sorted`

ONE OBJECT, MANY STEPS

Builder Pattern -- Visual Flow



Each box represents a step in the chain. Notice the SAME object (sb) flows through every `.append()` call -- nothing new is created at each step, we simply keep modifying and returning the same builder.

This exact 'flows through and returns itself' idea is what makes Stream method chains (like `.filter().map().sorted()`) possible.

5

SECTION 5

Introduction to Streams

What exactly is a Stream in Java 8?

TWO VERY DIFFERENT IDEAS

Collection vs Stream

Collection

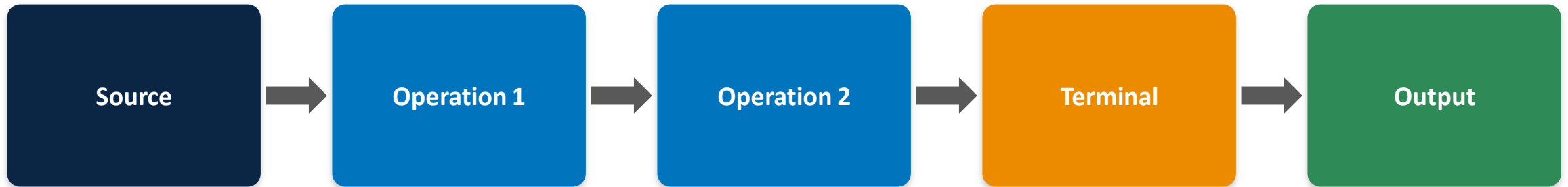
- A data structure that STORES elements
- Can be iterated many times
- You add/remove elements directly
- Exists in memory, holding data
- Example: ArrayList, HashSet

Stream

- A sequence of elements that supports PROCESSING
- Can be consumed (used) only ONCE
- You never add/remove -- only transform/filter
- Does not store data -- it flows through operations
- Example: `list.stream()`

DATA FLOWS THROUGH STEPS

The Stream Processing Pipeline



A Stream pipeline always has 3 parts: a SOURCE (usually a collection), zero or more OPERATIONS (filter, map, sorted...), and exactly one TERMINAL operation (forEach, collect, count...) that actually triggers the processing and produces a result.

PROGRAM

Your First Stream Program

Let's convert a Collection into a Stream and print every element using `forEach()`.

```
ArrayList<Integer> list = new ArrayList<>();  
  
list.add(10);  
list.add(20);  
list.add(30);  
  
list.stream().forEach(System.out::println);
```

OUTPUT

```
10  
20  
30
```

KEY POINTS

- ▶ `.stream()` converts the `ArrayList` into a `Stream`
- ▶ `.forEach()` is a **TERMINAL** operation -- it triggers execution
- ▶ `System.out::println` is a **Method Reference** (a short form of a `lambda`)
- ▶ Without a terminal operation, **NOTHING** happens -- `Streams` are lazy

6

SECTION 6

Lambda Expressions

The fuel that powers every Stream operation

A FUNCTION WITHOUT A NAME

What Is a Lambda Expression?

1

Definition

A short block of code that represents a function -- written inline, without needing a class or a method name.

2

Why it exists

Java 8 needed a compact way to pass behavior (a piece of logic) as an argument to methods like `filter()` and `map()`.

3

Basic syntax

(parameters) -> expression Example: `x -> x * 2`

4

Replaces old style

Before Java 8, we needed a full anonymous class just to pass one line of logic -- lambdas remove that boilerplate.

SAME LOGIC, LESS CODE

Traditional Method vs Lambda

Traditional (Anonymous Class)

- `new Runnable() { ... }`
- `public void run() { ... }`
- Multiple lines just to define one action
- Verbose and repetitive boilerplate
- Common before Java 8

Lambda (Java 8)

- `() -> System.out.println("Running")`
- Same logic in ONE line
- No class name, no method name needed
- Compiler infers everything from context
- The modern, preferred style

PROGRAM

Lambda in Action: forEach()

Instead of a traditional for loop, we can print every name using a lambda passed to forEach().

```
ArrayList<String> names = new ArrayList<>();
names.add("Raju");
names.add("Amit");

names.forEach(name -> System.out.println(name));
```

OUTPUT

```
Raju
Amit
```

KEY POINTS

- ▶ 'name' is the parameter -- it represents each element, one at a time
- ▶ -> separates the parameter from the logic to run
- ▶ No loop keyword, no index variable -- Java handles the iteration
- ▶ This can be shortened further to `names.forEach(System.out::println)`

7

SECTION 7

Iterating Through Collections

Four ways to walk through the same data

COMPARISON

Four Ways to Iterate a Collection

Technique	Style	Best For
Traditional for loop	<code>for(int i=0;i<list.size();i++)</code>	When you need the index
Enhanced for-each loop	<code>for(String s : list)</code>	Simple reading, no index needed
Iterator	<code>while(it.hasNext())</code>	Safe removal while iterating
Lambda <code>forEach()</code>	<code>list.forEach(s -> ...)</code>	Modern, concise, Java 8+ style

PROGRAM COMPARISON

Iteration Style 1 & 2: for-loop and for-each

Both loops below print the exact same output -- but the enhanced for-each is shorter and less error-prone.

```
// Traditional for loop
for (int i = 0; i < students.size(); i++) {
    System.out.println(students.get(i));
}

// Enhanced for-each loop
for (String s : students) {
    System.out.println(s);
}
```

OUTPUT

```
Raju
Amit
Neha
Raju
Amit
```

```
Neha
```

KEY POINTS

- ▶ Traditional loop needs an index variable `i` and `.get(i)`
- ▶ for-each loop reads each element directly as `s`
- ▶ for-each avoids index-out-of-bounds mistakes
- ▶ Both loops print the list twice here for comparison

PROGRAM COMPARISON

Iteration Style 3 & 4: Iterator and Lambda forEach

Iterator gives manual control (and safe removal); lambda forEach() is the shortest, most modern style.

```
// Using Iterator
Iterator<String> it = students.iterator();
while (it.hasNext()) {
    System.out.println(it.next());
}

// Using Lambda forEach()
students.forEach(s -> System.out.println(s));
```

OUTPUT

```
Raju
Amit
Neha
Raju
Amit
```

```
Neha
```

KEY POINTS

- ▶ hasNext() checks if more elements remain
- ▶ next() returns the current element and moves forward
- ▶ Iterator.remove() is the ONLY safe way to delete while looping
- ▶ forEach(lambda) is the cleanest, most Java-8-idiomatic style

8

SECTION 8

The Stream Interface

stream() vs parallelStream() and the Stream lifecycle

Understanding the Stream Interface

1

What it is

Stream<T> is an interface in java.util.stream that represents a sequence of elements supporting functional-style operations.

2

stream()

Creates a SEQUENTIAL stream -- elements are processed one after another, in order, on a single thread.

3

parallelStream()

Creates a PARALLEL stream -- elements are split across multiple CPU cores and processed simultaneously.

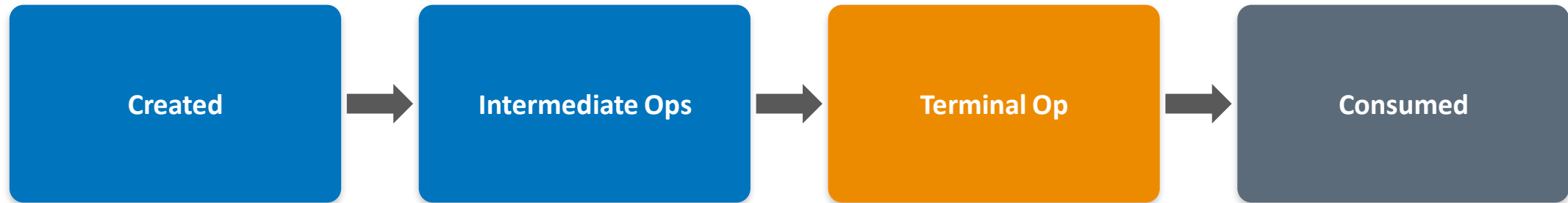
4

One-time use

Once a terminal operation runs on a Stream, that Stream is 'used up' and cannot be reused -- you must create a new one.

FROM CREATION TO RESULT

Stream Lifecycle



A Stream is **CREATED** from a source, passes through zero or more **INTERMEDIATE** operations (which return new streams), and finally runs exactly one **TERMINAL** operation, after which the stream is **CONSUMED** and can never be reused.

9

SECTION 9

Filtering With filter()

Selecting only the elements you want

SELECTING WHAT YOU NEED

The filter() Method & Predicate

1

filter()

An intermediate operation that keeps only the elements matching a given condition, and discards the rest.

2

Predicate

A functional interface representing a test that takes one input and returns true or false.

3

How it's used

`stream.filter(x -> x > 10)` -- keeps only elements greater than 10, in the same order.

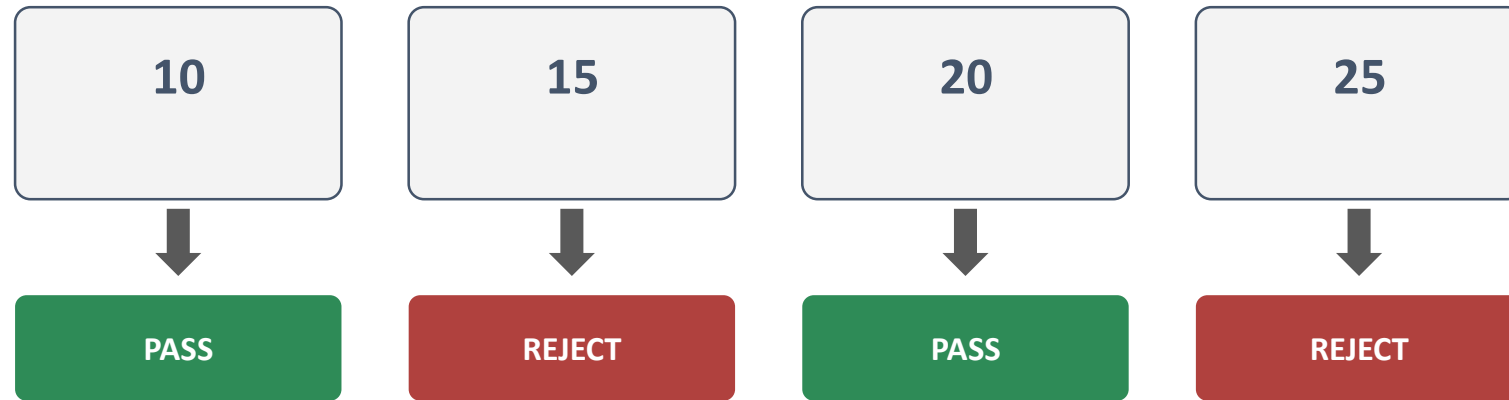
4

Lazy evaluation

`filter()` does not run immediately -- it only executes when a terminal operation is triggered.

ELEMENT BY ELEMENT

How filter() Works Internally



`filter(x -> x % 2 == 0)` tests EVERY element one at a time against the condition. 10 is even -> PASS. 15 is odd -> REJECT. 20 is even -> PASS. 25 is odd -> REJECT. Only the PASS elements continue down the pipeline to the next operation.

PROGRAM

filter() in Code

Let's filter only the even numbers from a list and print them.

```
list.stream()
    .filter(x -> x % 2 == 0)
    .forEach(System.out::println);
```

OUTPUT

```
10
20
30
```

KEY POINTS

- ▶ `x % 2 == 0` checks if a number is even (no remainder)
- ▶ `filter()` returns a NEW stream containing only matching elements
- ▶ Odd numbers are silently dropped -- not stored anywhere
- ▶ `forEach()` is still required to actually print the results

10

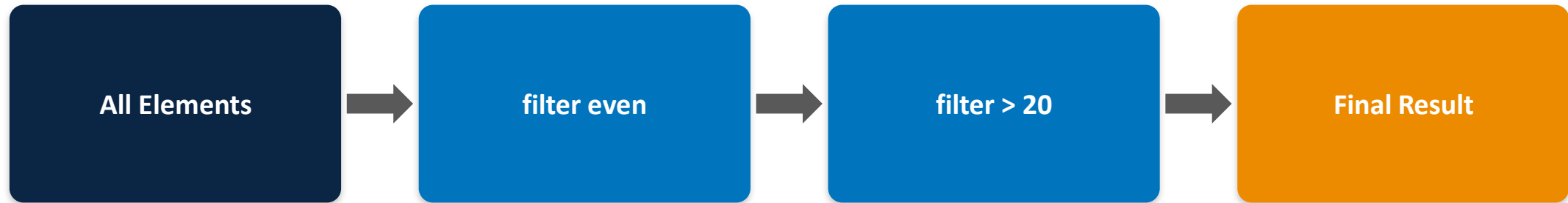
SECTION 10

Chaining Multiple Filters

Applying more than one condition

NARROWING DOWN STEP BY STEP

Multiple Filters -- Data Flow



Each filter() call narrows the stream further. First we keep only even numbers, THEN from those survivors we keep only the ones greater than 20. Order matters conceptually, though for simple conditions like these the final result is the same either way.

PROGRAM

Chaining Two Filters

We now chain TWO filter() calls: first keep even numbers, then keep only those greater than 20.

```
list.stream()
    .filter(x -> x % 2 == 0)
    .filter(x -> x > 20)
    .forEach(System.out::println);
```

OUTPUT

```
30
40
```

KEY POINTS

- ▶ Each .filter() call returns a new Stream to keep chaining on
- ▶ An element must pass BOTH conditions to reach forEach()
- ▶ This is equivalent to filter(x -> x % 2 == 0 && x > 20)
- ▶ Chaining keeps each condition small, readable, and testable

11

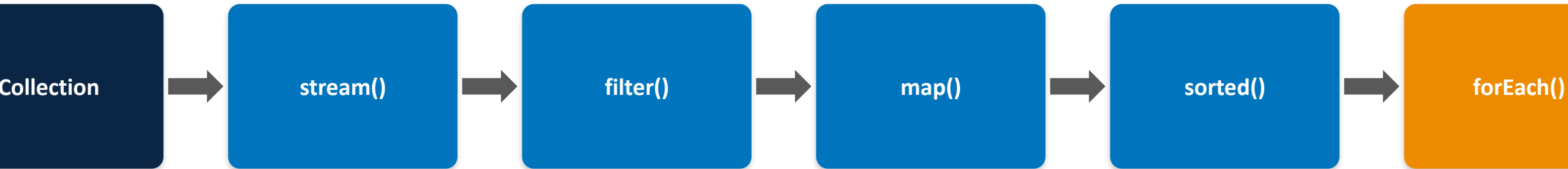
SECTION 11

Method Chaining in Streams

How one operation flows into the next

COLLECTION TO CONSOLE

The Full Method Chain



Every arrow in this chain represents one method call returning a NEW Stream object, which is why we can immediately call the next method on it -- exactly like the StringBuilder Builder Pattern we studied earlier.

PROGRAM

Method Chaining -- Full Example

Here is a complete chain: filter, transform with map, sort, then print -- all in one readable statement.

```
list.stream()  
    .filter(x -> x > 10)  
    .map(x -> x * 2)  
    .sorted()  
    .forEach(System.out::println);
```

OUTPUT

```
(sample) 40  
60  
80
```

KEY POINTS

- ▶ filter() keeps elements greater than 10
- ▶ map() doubles each surviving element
- ▶ sorted() arranges results in ascending order
- ▶ forEach() prints the final, fully processed result

12

SECTION 12

The map() Method

Transforming elements, not selecting them

SELECTING VS TRANSFORMING

filter() vs map()

filter()

- Decides WHICH elements to keep
- Input and output types are the SAME
- Number of elements can DECREASE
- Uses a Predicate (returns true/false)

map()

- Decides HOW to transform each element
- Input and output types CAN differ
- Number of elements stays the SAME
- Uses a Function (returns a new value)

PROGRAM

map() in Code -- Before and After

map() applies a transformation to EVERY element and produces a new Stream of transformed values.

```
list.stream()
    .map(x -> x * 10)
    .forEach(System.out::println);
```

OUTPUT

```
Before: 1, 2, 3
After:  10, 20, 30
```

KEY POINTS

- ▶ Every element is multiplied by 10 -- none are dropped
- ▶ The stream size stays exactly the same, only values change
- ▶ map() can also change the TYPE, e.g. Integer to String
- ▶ map() is an intermediate operation -- it can be chained further

13

SECTION 13

The Stream Pipeline

Source, Intermediate Operations, Terminal Operation

THREE ESSENTIAL PARTS

Anatomy of a Stream Pipeline



SOURCE: where data comes from (a Collection, an array, etc.)

INTERMEDIATE OPERATIONS: filter, map, sorted, distinct, limit, skip -- these return a Stream so they can be chained, and are LAZY (they don't run until needed).

TERMINAL OPERATION: forEach, collect, count, reduce -- exactly ONE per pipeline, and it triggers all the work to actually happen.

PROGRAM

A Complete Stream Pipeline

Let's identify Source, Intermediate Operations, and Terminal Operation in one real pipeline.

```
list.stream()
    .filter(x -> x > 10)
    .map(x -> x * 2)
    .sorted()
    .forEach(System.out::println);
```

OUTPUT

(depends on input list, printed in ascending order)

KEY POINTS

- ▶ Source: list.stream()
- ▶ Intermediate Operations: filter(), map(), sorted()
- ▶ Terminal Operation: forEach() -- triggers the whole pipeline
- ▶ Without forEach(), NOTHING in this chain would execute at all

14

SECTION 14

Intermediate Operations

filter, map, sorted, distinct, limit, skip

SIX KEY OPERATIONS

Intermediate Operations at a Glance

Operation	Purpose	Example
<code>filter(Predicate)</code>	Keep matching elements	<code>filter(x -> x > 5)</code>
<code>map(Function)</code>	Transform each element	<code>map(x -> x * 2)</code>
<code>sorted()</code>	Sort in natural order	<code>sorted()</code>
<code>distinct()</code>	Remove duplicate values	<code>distinct()</code>
<code>limit(n)</code>	Keep only first n elements	<code>limit(3)</code>
<code>skip(n)</code>	Skip the first n elements	<code>skip(2)</code>

PROGRAM

distinct() -- Removing Duplicates

distinct() removes duplicate values from the stream, keeping only the first occurrence of each.

```
List<Integer> nums = Arrays.asList(1, 2, 2, 3, 3, 3, 4);

nums.stream()
    .distinct()
    .forEach(System.out::println);
```

OUTPUT

```
1
2
3
4
```

KEY POINTS

- ▶ distinct() uses equals() to detect duplicate values
- ▶ Only the FIRST occurrence of each value is kept
- ▶ Original order of first-occurrences is preserved
- ▶ Very useful before further processing to avoid double-counting

PROGRAM

limit() and skip() -- Slicing a Stream

limit(n) takes only the first n elements; skip(n) discards the first n and keeps the rest.

```
List<Integer> nums = Arrays.asList(10,20,30,40,50,60);

nums.stream().limit(3).forEach(System.out::println);
System.out.println("---");
nums.stream().skip(3).forEach(System.out::println);
```

OUTPUT

```
10
20
30
---
40
50
```

KEY POINTS

- ▶ limit(3) keeps ONLY the first 3 elements
- ▶ skip(3) removes the first 3, keeps everything after
- ▶ Together they are often used for pagination (page 2, page 3...)
- ▶ Both are intermediate operations -- they can be combined with others

15

SECTION 15

Terminal Operations

forEach, collect, count, reduce, and matching operations

THE OPERATIONS THAT TRIGGER EXECUTION

Terminal Operations at a Glance

Operation	Returns	Purpose
<code>forEach(Consumer)</code>	<code>void</code>	Perform an action on each element
<code>collect(Collector)</code>	<code>List/Set/Map...</code>	Gather results into a collection
<code>count()</code>	<code>long</code>	Count the number of elements
<code>reduce(...)</code>	single value	Combine all elements into one result
<code>findFirst() / findAny()</code>	<code>Optional<T></code>	Get one matching element
<code>anyMatch() / allMatch()</code>	<code>boolean</code>	Test if elements match a condition

PROGRAM

collect() and count()

collect() gathers the stream's results back into a real collection like a List.

```
List<Integer> result = list.stream()
    .filter(x -> x > 10)
    .collect(Collectors.toList());

System.out.println(result);
System.out.println(result.size());

long total = list.stream().filter(x -> x > 10).count();
System.out.println(total);
```

OUTPUT

```
[20, 30, 40]
3
3
```

KEY POINTS

- ▶ Collectors.toList() is the most common collector
- ▶ collect() turns a temporary Stream back into a permanent List
- ▶ count() returns how many elements passed through as a long
- ▶ Use count() when you only need the number, not the elements

PROGRAM

reduce(), findFirst(), and Matching Operations

These terminal operations answer specific questions: one value, one element, or a yes/no answer.

```
int sum = list.stream().reduce(0, (a, b) -> a + b);

Optional<Integer> first = list.stream().findFirst();

boolean anyBig = list.stream().anyMatch(x -> x > 100);
boolean allPositive = list.stream().allMatch(x -> x > 0);
```

OUTPUT

```
sum = 60
first = Optional[10]
anyBig = false
allPositive = true
```

KEY POINTS

- ▶ reduce() combines everything into ONE final value (details next section)
- ▶ findFirst() returns the first element wrapped in an Optional
- ▶ anyMatch() -- true if AT LEAST ONE element matches
- ▶ allMatch() -- true only if EVERY element matches

16

SECTION 16

Parallel Streams

Using multiple CPU cores to process data faster

ONE CORE VS MANY CORES

Sequential Stream vs Parallel Stream

Sequential Stream (stream())

- Processes elements one at a time, in order
- Runs on a single thread
- Predictable, easy to debug
- Best for small or simple datasets

Parallel Stream (parallelStream())

- Splits data across multiple CPU cores
- Runs on several threads simultaneously
- Order of output CAN vary
- Best for large datasets with heavy computation

PROGRAM

parallelStream() in Action

The same forEach() call, but now Java may use multiple threads to process elements simultaneously.

```
list.parallelStream()  
    .forEach(System.out::println);
```

OUTPUT

```
(Order may vary each run! e.g.)  
30  
10  
20
```

KEY POINTS

- ▶ Java's Fork/Join framework splits the work across CPU cores
- ▶ Each core may finish at a different time -- so ORDER is not guaranteed
- ▶ Never rely on parallelStream() when order matters
- ▶ For large collections, parallel streams can run noticeably faster

17

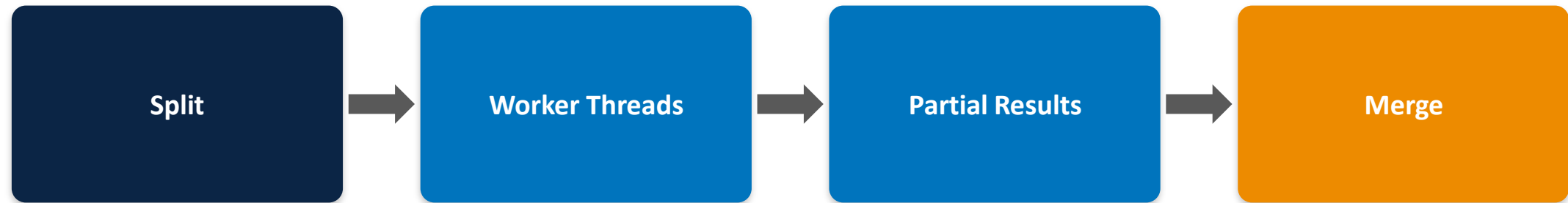
SECTION 17

Decomposing and Merging Work

How parallel streams split work and combine results (Fork/Join)

THE FORK/JOIN IDEA

Splitting and Merging Work



SPLIT: the large task is broken into smaller chunks.

WORKER THREADS: each chunk is processed independently and simultaneously by a different CPU core/thread.

PARTIAL RESULTS: each worker produces its own small result.

MERGE: all partial results are combined together into the final answer.

THE ENGINE BEHIND PARALLEL STREAMS

Fork/Join Framework -- Beginner Explanation

1

Fork

A big task is split ('forked') into smaller sub-tasks that can be solved independently.

2

Multiple workers

Java's thread pool assigns each sub-task to an available CPU core to work on simultaneously.

3

Join

Once all sub-tasks finish, their results are combined ('joined') back into one final result.

4

Why it matters

This is exactly what happens automatically behind the scenes whenever you call `.parallelStream()`.

18

SECTION 18

Reduction

Combining every element into a single final value

MANY VALUES BECOME ONE

What Is Reduction?

1

Definition

Reduction combines all elements of a Stream into a single result, by repeatedly applying a combining operation.

2

Why we need it

Common tasks like sum, maximum, minimum, or joining text all follow this exact 'combine everything' pattern.

3

The reduce() method

`Stream.reduce(identity, accumulator)` -- identity is the starting value, accumulator is the combining logic.

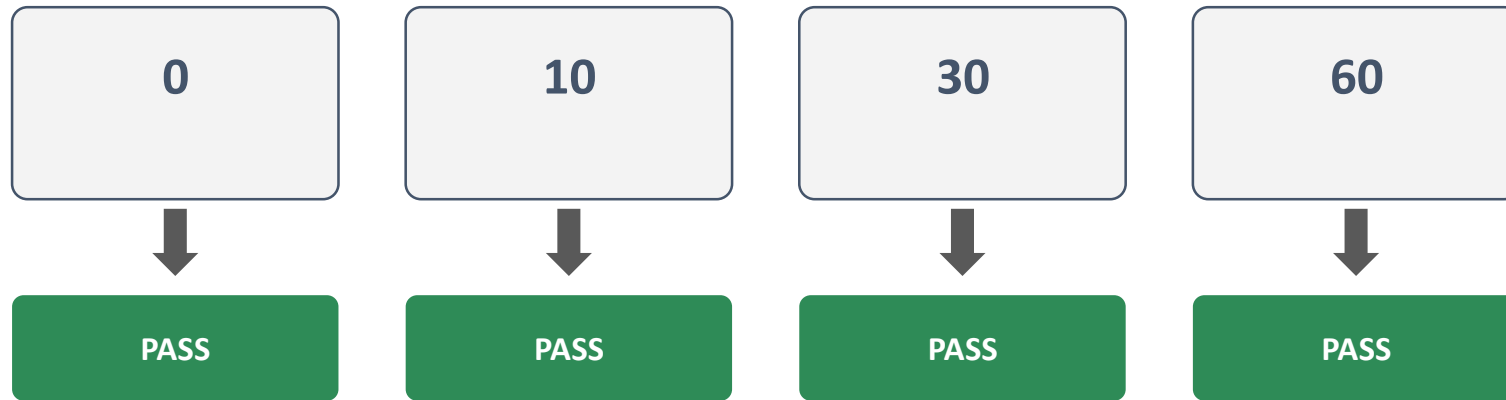
4

Accumulator

A function that takes two values (running result so far, and next element) and produces a new running result.

STEP-BY-STEP ACCUMULATION

How reduce() Works -- Running Total



Starting value (identity) = 0. Add 10 -> running total becomes 10. Add 20 -> running total becomes 30. Add 30 -> running total becomes 60. This running total, updated one element at a time, is exactly what the accumulator function does inside `reduce()`.

PROGRAM

Calculating a Value Using reduce()

Let's compute the sum of a list using reduce() with an identity of 0 and an addition accumulator.

```
int sum = list.stream()
    .reduce(0, (a, b) -> a + b);

System.out.println(sum);
```

OUTPUT

60

KEY POINTS

- ▶ 0 is the identity -- the starting value if the list were empty
- ▶ (a, b) -> a + b is the accumulator: a = running total, b = next element
- ▶ reduce() applies this accumulator repeatedly across the whole stream
- ▶ The FINAL running total becomes the single returned result

19

SECTION 19

More reduce() Examples

Maximum, Minimum, Product, and String Concatenation

PROGRAM

Maximum and Minimum Using reduce()

We can find the maximum or minimum value in a stream by changing only the accumulator logic.

```
int max = list.stream()
    .reduce(Integer.MIN_VALUE, (a,b) -> a > b ?
a : b);

int min = list.stream()
    .reduce(Integer.MAX_VALUE, (a,b) -> a < b ?
a : b);

System.out.println("Max: " + max);
System.out.println("Min: " + min);
```

OUTPUT

```
Max: 30
Min: 10
```

KEY POINTS

- ▶ Integer.MIN_VALUE is a safe starting point for finding a maximum
- ▶ The ternary `a > b ? a : b` keeps whichever value is larger
- ▶ The exact same `reduce()` shape, only the accumulator logic changes
- ▶ Java also offers `list.stream().max(Comparator.naturalOrder())` as a shortcut

PROGRAM

Product and String Concatenation Using reduce()

reduce() can multiply numbers together, or join strings -- the pattern is always identity + accumulator.

```
int product = list.stream()
    .reduce(1, (a,b) -> a * b);

String combined = names.stream()
    .reduce("", (a,b) -> a + " " + b);

System.out.println(product);
System.out.println(combined.trim());
```

OUTPUT

```
6000
Raju Amit Neha
```

KEY POINTS

- ▶ Identity for multiplication must be 1 (not 0!) -- else result is always 0
- ▶ Identity for string joining is an empty string ""
- ▶ The accumulator defines exactly HOW two values combine
- ▶ This proves reduce() is a general pattern, reused for many purposes

20

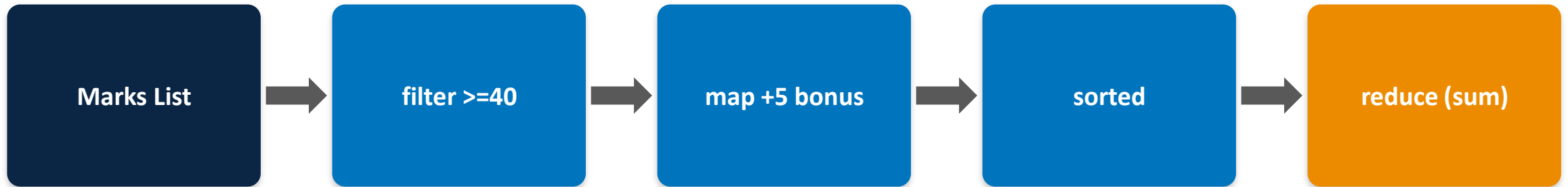
SECTION 20

A Complete Real-World Pipeline

Student marks: filter, map, sort, and reduce together

EVERYTHING COMES TOGETHER

Real-World Pipeline: Student Marks



SCENARIO: A teacher has a list of student marks and wants the total of all PASSING students (marks ≥ 40), after adding a 5-mark bonus to each, sorted for a clean printed report before summing.

FULL PROGRAM

Complete Pipeline: Student Marks Example

This single pipeline filters passing students, adds bonus marks, sorts them, and prints the total using reduce().

```
List<Integer> marks = Arrays.asList(35, 42, 58, 39, 91, 40);

List<Integer> passed = marks.stream()
    .filter(m -> m >= 40)
    .map(m -> m + 5)
    .sorted()
    .collect(Collectors.toList());

System.out.println("Sorted results: " + passed);

int total = passed.stream().reduce(0, (a,b) -> a + b);
System.out.println("Total: " + total);
```

OUTPUT

```
Sorted results: [45, 47, 63, 96]
Total: 251
```

KEY POINTS

- ▶ `filter(m -> m >= 40)` removes failing students first
- ▶ `map(m -> m + 5)` adds the bonus to each passing mark
- ▶ `sorted() + collect()` produces a clean, ordered List to display
- ▶ A second small pipeline uses `reduce()` to compute the final total

21

SECTION 21

Performance: Sequential vs Parallel

Choosing the right approach for the job

PRACTICAL GUIDELINES

When to Use Each Approach

Use Sequential Stream When...

- The dataset is small (a few hundred items or fewer)
- Operations are simple and fast
- Order of processing/output matters
- You need simple, predictable debugging

Use Parallel Stream When...

- The dataset is large (thousands+ items)
- Each operation is computationally expensive
- Order does NOT matter for the result
- Your machine has multiple CPU cores available

WRITING PROFESSIONAL STREAM CODE

Best Practices for Streams

1

Keep lambdas short

If a lambda needs many lines, extract it into a named method and use a method reference instead.

2

Avoid side effects

Don't modify external variables inside filter/map -- keep Stream operations 'pure' and predictable.

3

Don't overuse parallel

Measure performance before assuming `parallelStream()` is faster -- test, don't guess.

4

Prefer method references

`System.out::println` is cleaner than `x -> System.out.println(x)` when no extra logic is needed.

22

SECTION 22

Common Interview Questions

Be ready to explain these clearly and confidently

CORE DISTINCTIONS

Interview Questions -- Part 1

Question	Key Answer
Collection vs Stream?	Collection stores data; Stream processes data and can be used only once
filter() vs map()?	filter selects elements (Predicate); map transforms elements (Function)
stream() vs parallelStream()?	Sequential = single thread; Parallel = multiple threads, order may vary
reduce() vs collect()?	reduce() -> single combined value; collect() -> a full collection of results

DEEPER CONCEPTS

Interview Questions -- Part 2

Question	Key Answer
Intermediate vs Terminal ops?	Intermediate returns a Stream (lazy, chainable); Terminal triggers execution & produces a result
What is Lazy Evaluation?	Intermediate operations don't run until a terminal operation is called
What is the Builder Pattern?	A pattern where methods return the same/related object, enabling method chaining
What is Fork/Join?	A framework that splits (forks) work across threads and combines (joins) the results

23

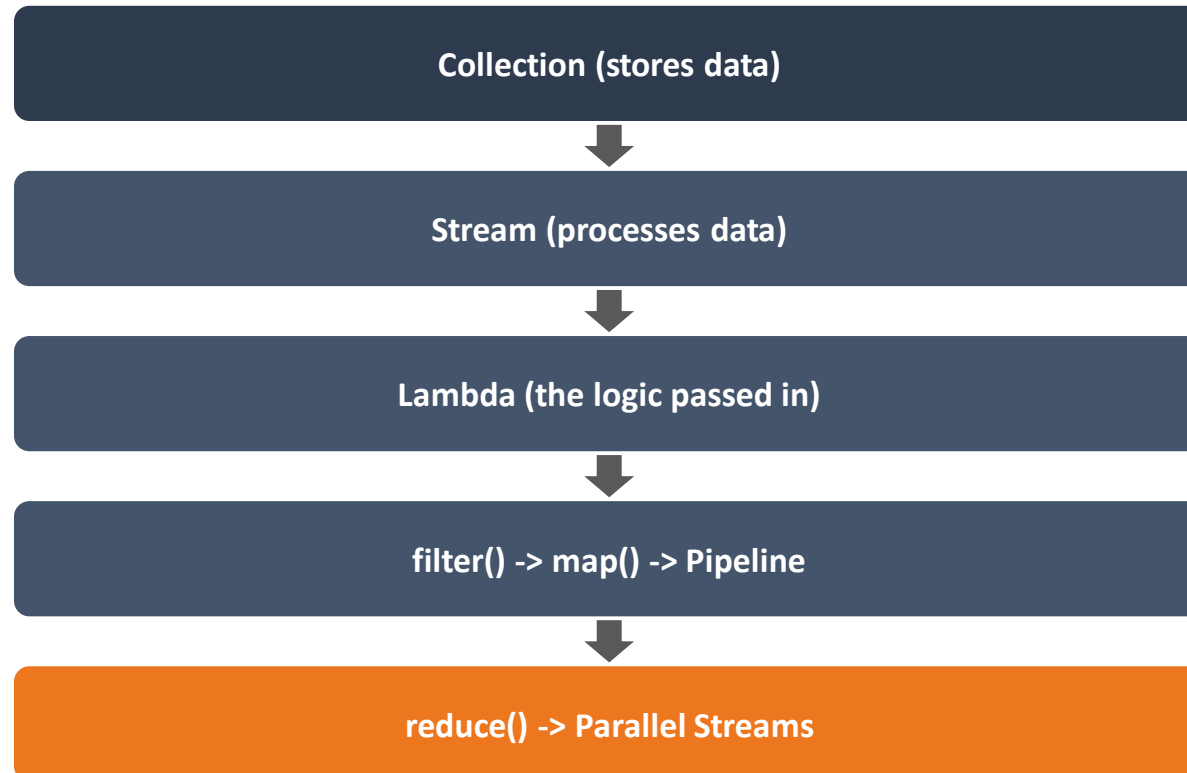
SECTION 23

Summary

One mind map tying every concept together

THE COMPLETE JOURNEY

Course Mind Map



Read this mind map top to bottom as the story of the whole course:

We start with a Collection that stores our data.
We convert it into a Stream to process it.
We write Lambda expressions as the logic for each step.
We chain filter() and map() into a Pipeline.
Finally we reduce() everything into one answer -- optionally in Parallel across CPU cores.

24

SECTION 24

Practice Programs

15 exercises -- from beginner to advanced

TRY THESE YOURSELF

15 Practice Exercises -- Part A

1. Print all even numbers from a list using `filter()`
2. Square every number in a list using `map()`
3. Remove duplicate values from a list using `distinct()`
4. Sort a list of names alphabetically using `sorted()`
5. Find the maximum number in a list using `reduce()`
6. Find the sum of all numbers in a list using `reduce()`
7. Print only students who scored above 80 marks

Continued on next slide...

TRY THESE YOURSELF

15 Practice Exercises -- Part B

1. Convert all names in a list to uppercase using `map()`
2. Calculate the average of all marks in a list
3. Count how many numbers in a list are odd
4. Find the first name that starts with the letter 'A'
5. Check if any number in a list is negative (`anyMatch`)
6. Check if all numbers in a list are positive (`allMatch`)
7. Join all student names into one comma-separated String
8. Print only the first 3 elements of a list using `limit()`

Solutions can be reviewed together in the next class session.

TRY THESE YOURSELF

15 Practice Exercises -- Part B

1. Convert all names in a list to uppercase using `map()`
2. Calculate the average of all marks in a list
3. Count how many numbers in a list are odd
4. Find the first name that starts with the letter 'A'
5. Check if any number in a list is negative (`anyMatch`)
6. Check if all numbers in a list are positive (`allMatch`)
7. Join all student names into one comma-separated String
8. Print only the first 3 elements of a list using `limit()`

Solutions can be reviewed together in the next class session.

TRY THESE YOURSELF

15 Practice Exercises -- Part B

1. Convert all names in a list to uppercase using `map()`
2. Calculate the average of all marks in a list
3. Count how many numbers in a list are odd
4. Find the first name that starts with the letter 'A'
5. Check if any number in a list is negative (`anyMatch`)
6. Check if all numbers in a list are positive (`allMatch`)
7. Join all student names into one comma-separated String
8. Print only the first 3 elements of a list using `limit()`

Solutions can be reviewed together in the next class session.

Intermediate

`filter()`

`map()`

`sorted()`

`distinct()`

`limit()`

`skip()`

`peek()`

`mapToInt()`

Terminal

`forEach()`

`collect()`

`reduce()`

`count()`

`min()`

`max()`

`findFirst()`

`findAny()`

`anyMatch()`

`allMatch()`

`noneMatch()`



Thank You

Questions & Discussion

Practice every program on your own IDE -- Streams are learned by typing, not just reading.